Acknowledgements & Copyright Notice

© 2026 Dr. Pandarasamy Arjunan, Indian Institute of Science, Bangalore. All rights reserved.

These slides are distributed under a Creative Commons License. See Creative Commons Attribution-ShareAlike 2.0 License.

The author makes these slides available for educational purposes. They may not be used or distributed for commercial purposes. You may copy, use, or distribute these slides for educational purposes, provided you cite the author as the source.

Credits: This presentation includes slides and images from various books, research articles, and online sources, including:

- NVIDIA's Edge AI and Robotics Teaching Kit
- Harvard - TinyML Courseware and Machine Learning Systems
- MIT - TinyML and Efficient Deep Learning Computing
- Dive into Deep Learning and Luis Serrano's Academy and his books and videos
- IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things, 2017 by David Hanes
- Introduction to IoT, 2021 by Anandarup Mukherjee; *AI at the Edge* by Daniel Situnayake, Jenny Plunkett - O'Reilly

The author acknowledges and thanks the creators for their contributions. Images and content are used under fair use for educational purposes.

Disclaimer: All logos, images, and other trademarks are the property of their respective owners. If you believe that any content in this presentation violates copyright laws, please contact samy@iisc.ac.in to have it promptly removed or appropriately attributed.

Instruction Enhancement Program (IEP) on

Edge AI

Lab 1: Micro Python

Pandarasamy Arjunan (Samy)
Assistant Professor
Robert Bosch Centre for Cyber Physical Systems
Indian Institute of Science, Bengaluru
samy@iisc.ac.in

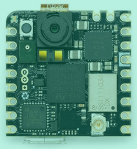
Image source: https://leverx.com

Lab Overview

1. Overview of Arduino Nicla Vision
2. Introduction to MicroPython programming
3. Software installation and testing
   - OpenMV IDE setup
   - Reading IMU data
   - Reading Camera data

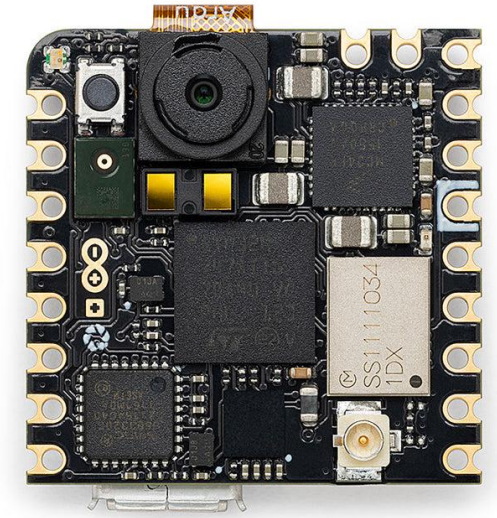
Part 1: Overview of Arduino Nicla Vision

Embedded Platforms

	Board	MCU / ASIC	Clock	Memory	Sensors	Radio
	Himax WE-I Plus EVB	HX6537-A 32-bit EM9D DSP	400 MHz	2MB flash 2MB RAM	Accelerometer, Mic, Camera	None
	Arduino Nano 33 BLE Sense	32-bit nRF52840	64 MHz	1MB flash 256kB RAM	Mic, IMU, Temp, Humidity, Gesture, Pressure, Proximity, Brightness, Color	BLE
	SparkFun Edge 2	32-bit ArtemisV1	48 MHz	1MB flash 384kB RAM	Accelerometer, Mic, Camera	BLE
	Espressif EYE	32-bit ESP32-D0WD	240 MHz	4MB flash 520kB RAM	Mic, Camera	WiFi, BLE
	Arduino Nicla Vision	Dual Arm® Cortex® M7/M4	240 MHz 480 MHz	2MB Flash 1MB RAM	6-Axis IMU 2 MP Color Camera Distance / Time Of Flight sensor Microphone	WiFI, BLE

Arduino Nicla Vision

Arduino Nicla Vision



Microcontroller	•STM32H747AII6 Dual Arm® Cortex® M7/M4 IC:1x Arm® Cortex® M7 core up to 480 MHz •1x Arm® Cortex® M4 core up to 240 MHz
Sensors	•2 MP Color Camera •6-Axis IMU (LSM6DSOX) •Distance / Time Of Flight sensor (VL53L1CBV0FY/1) •Microphone (MP34DT05)
Power	•3.7V Li-po battery with Integrated battery charger and fuel gauge (MAX17262REWL)
Memory	2MB Flash / 1MB RAM 16MB QSPI Flash for storage
Connectivity	Wi-Fi / Bluetooth® Low Energy 4.2 (Murata 1DX - LBEE5KL1DX-883)

Source: https://docs.arduino.cc/hardware/nicla-vision/

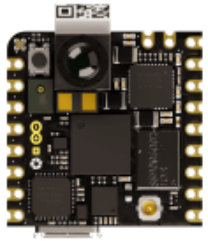
Datasheet

- Read Nicla Vision Datasheet and understand its features

PRO

Arduino® Nicla Vision

User Manual
SKU: ABX00051



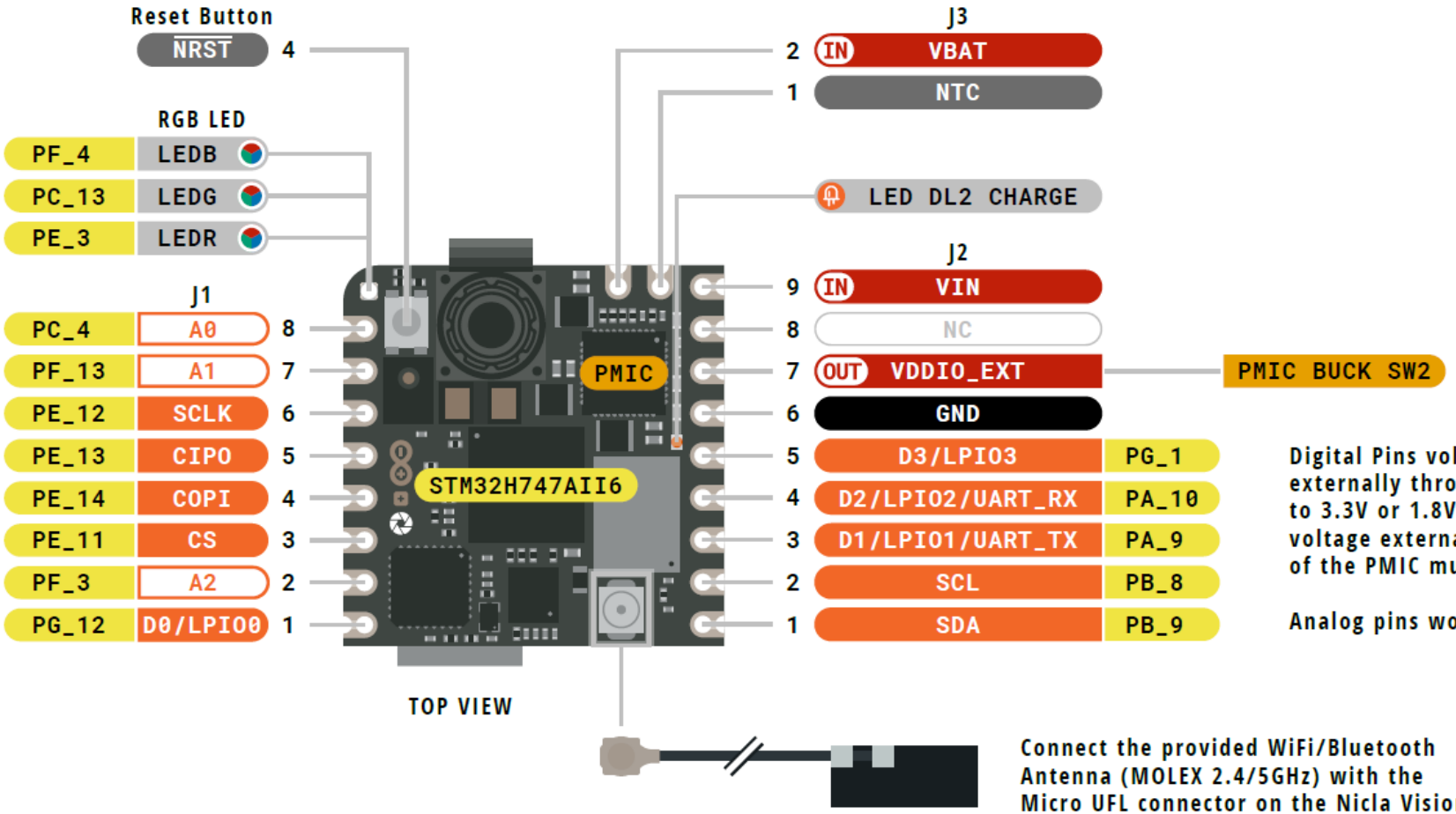
Description

The **Arduino® Nicla Vision** packs machine vision capabilities on the edge into a tiny fingerprint. Record, analyze and upload to the Cloud with the help of one **Arduino® Nicla Vision**. Leverage the onboard camera, STM32 microcontroller, Wi-Fi®/Bluetooth® module and 6-axis IMU to create your wireless sensor network for machine vision applications.

Target Areas

Wireless sensor networks, data fusion, artificial intelligence, machine vision

Pins



Digital Pins voltage level can be forced externally through VDDIO_EXT or internally set to 3.3V or 1.8V by software (to force the voltage externally, the buck converter no. 2 of the PMIC must be shut down)

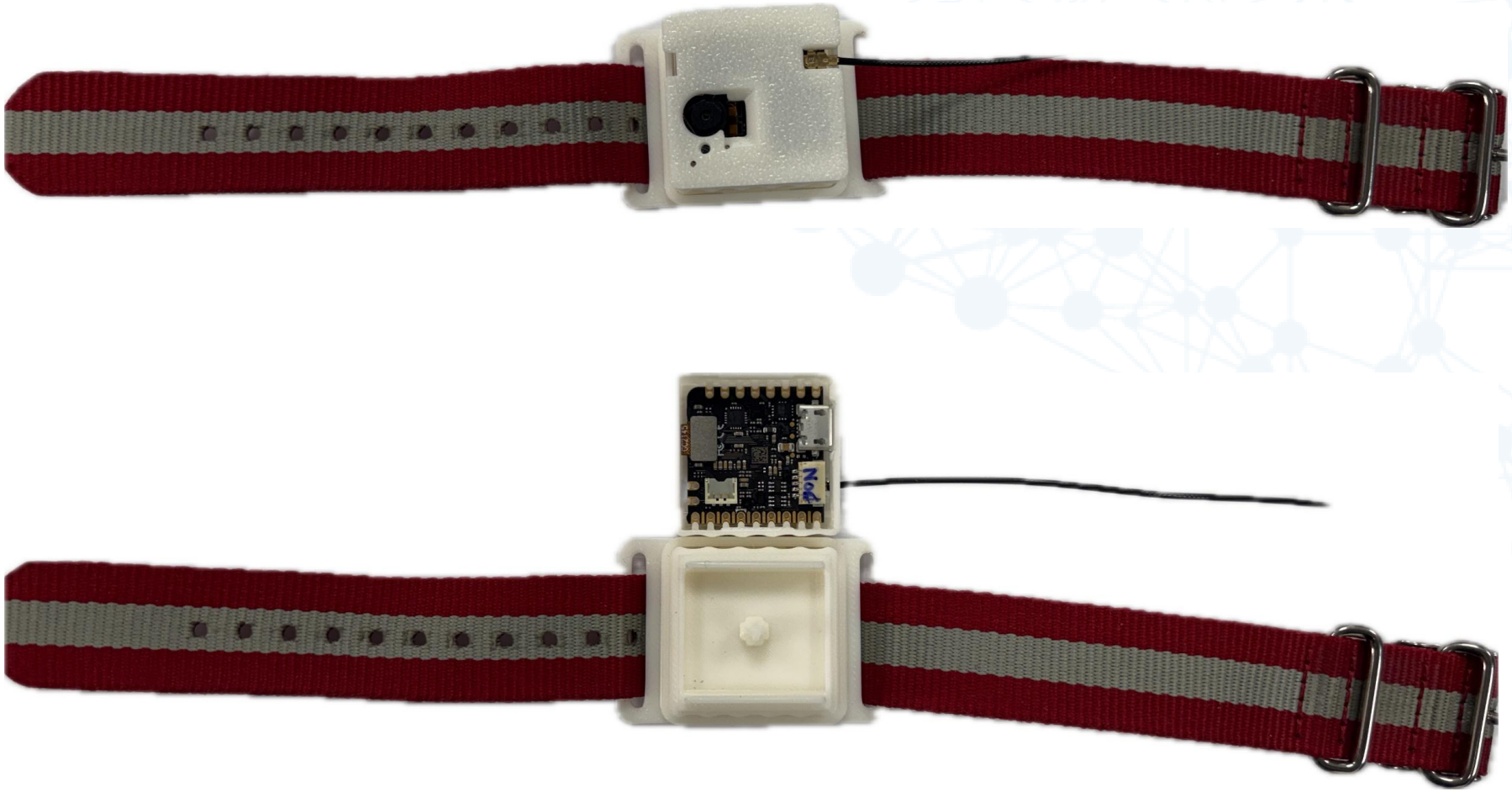
Analog pins work only at 1.8V

Connect the provided WiFi/Bluetooth Antenna (MOLEX 2.4/5GHz) with the Micro UFL connector on the Nicla Vision

Legend:

- Power
- Power Input
- Power Output
- Ground
- GPIO Digital External
- Analog External
- Main Part
- Secondary Part
- Internal Component
- Other Pins (Reset, System Control, Debugging)
- LED
- RGB LED
- Other

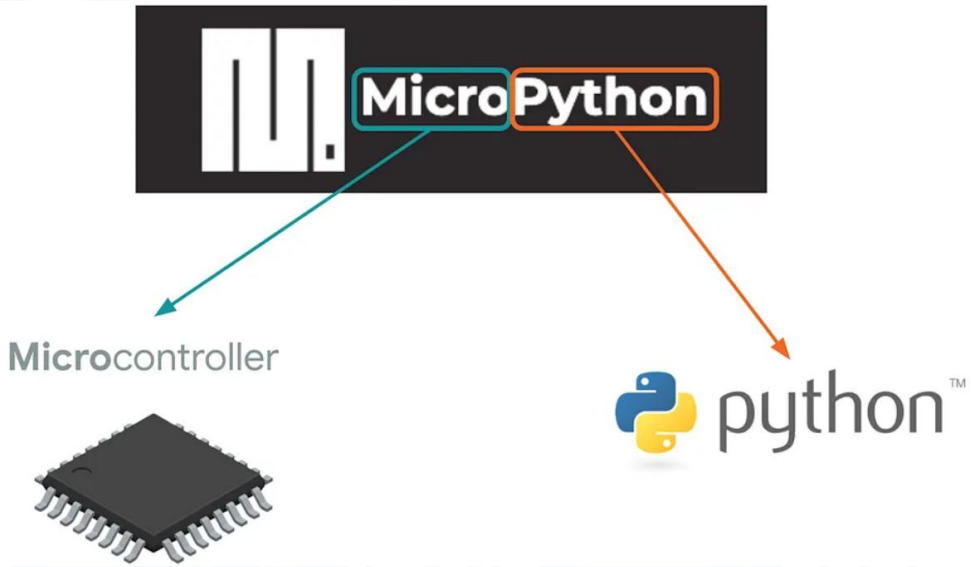
Our Smart Wearable Kit using Nicla Vision



Part 2: Introduction to MicroPython

MicroPython

- MicroPython is a lightweight implementation of Python 3 designed for microcontrollers and embedded systems.
- On Arduino boards, MicroPython firmware is first installed on the device, enabling it to interpret and execute Python code.
- Unlike traditional workflows that require compiling and flashing code, MicroPython allows you to write and run Python scripts directly on the board.
- This enables interactive development and faster prototyping on embedded hardware.
- MicroPython is supported on 100+ microcontroller boards
- Across 10+ MCU families (e.g., STM32, ESP32/ESP8266, RP2040, nRF52, SAMD, etc.)



Why Nicla Vision + MicroPython?

- Ideal for Python-first embedded development without writing low-level C/C++
- Powerful MCU, camera, microphone, and IMU
- Python productivity with near bare-metal hardware access
- Enables rapid prototyping for TinyML, Embedded Computer Vision, and Sensor-based applications

How does MicroPython work?

- **File System**: A lightweight file system on the microcontroller that stores Python scripts and configuration files.
- **main.py**: Executes automatically on-board boot and contains the main application logic.
- **boot.py**: Runs before main.py and is used for low-level configuration (e.g., Wi-Fi setup, hardware initialization).
- Base Modules:
  - Built-in modules for hardware interaction (e.g., GPIO, sensors, communication interfaces).
  - Core modules include machine, network, and time.

MicroPython vs. C++

- **Ease of Use**: Programs are written in Python, making development faster and more accessible.
- **Real-Time Interactivity**: Code can be written, uploaded, and tested immediately without a compile step.
- **Resource Efficiency**: C++ is more efficient in terms of memory usage and execution speed.
- **Trade-off**: MicroPython prioritizes ease of development over peak performance, while still handling common hardware tasks effectively.

MicroPython Variants

Standard MicroPython - https://micropython.org/

- Official MicroPython project
- Runs on many MCUs (ESP32, STM32, RP2040, etc.)
- Focus: general-purpose embedded Python
- Provides core modules like machine, network, time

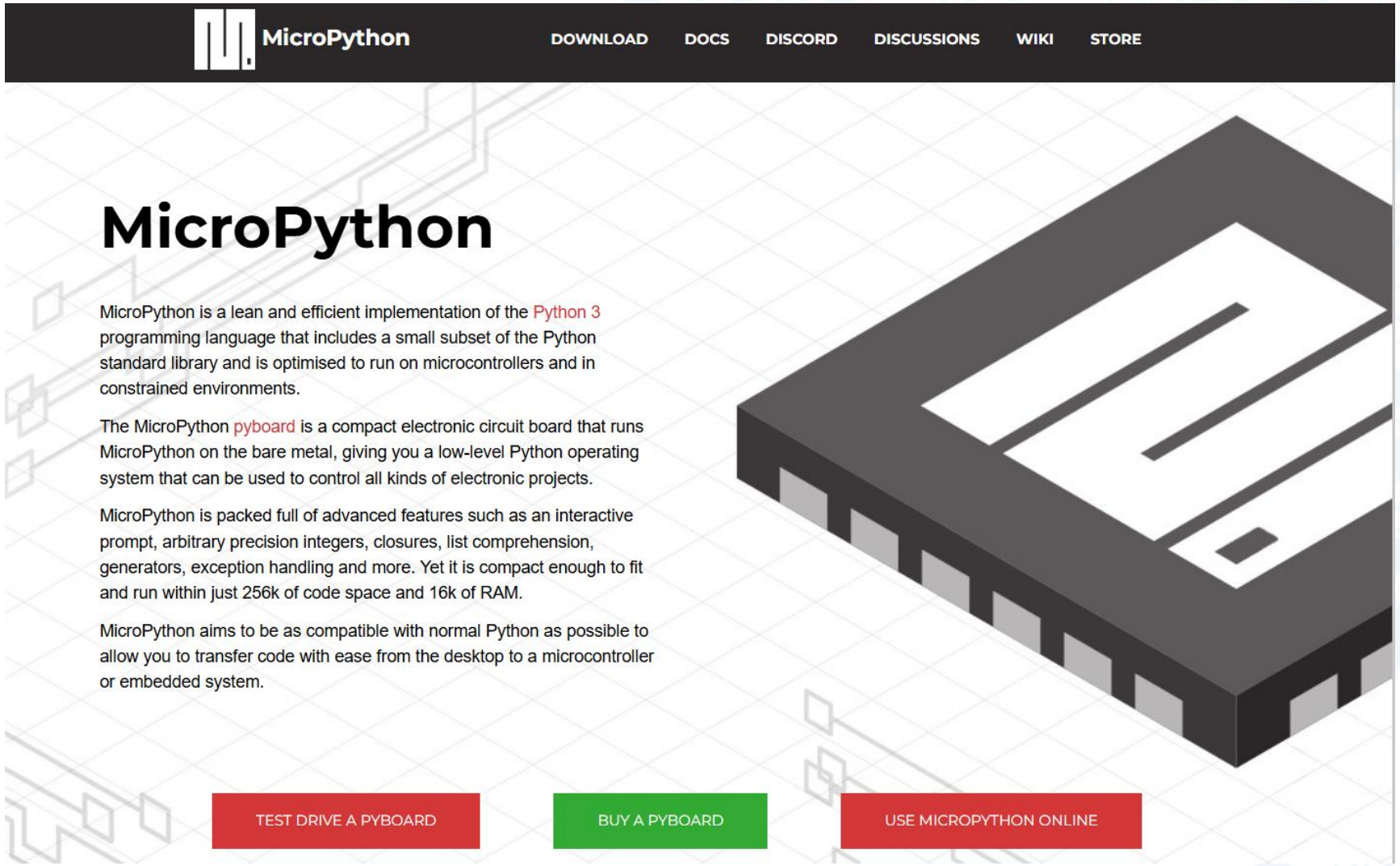
OpenMV MicroPython - https://docs.openmv.io/

- Customized MicroPython used by OpenMV boards and Nicla Vision
- Optimized for computer vision and sensors
- Includes precompiled native modules: sensor, image, imu
- Tight integration with OpenMV IDE

CircuitPython - https://circuitpython.org/

- Maintained by Adafruit
- Beginner-friendly MicroPython variant
- Emphasis on plug-and-play hardware, USB mass storage
- Slower than MicroPython, but easier for learning

Standard MicroPython



https://micropython.org/

OpenMV MicroPython

MicroPython

Search docs

MicroPython libraries

MicroPython language and implementation

MicroPython differences from CPython

MicroPython license information

Quick reference for the openmvcam

/ Overview

MicroPython documentation

Welcome! This is the documentation for MicroPython v1.26, for the OpenMV Cam and compatible platforms, last updated 28 Dec 2025.

General documentation for MicroPython:

Library Reference
MicroPython libraries and modules

Language Reference
information about MicroPython specific language features

MicroPython Differences
MicroPython operations which differ from CPython

License
MicroPython license information

References and tutorials:

Quick reference for the OpenMV Cam
pinout for the OpenMV Cam, snippets of useful code, and a tutorial

Indices and tables:

https://docs.openmv.io/

OpenMV MicroPython

- Vision-optimized MicroPython runtime
- Built-in camera, IMU, and peripheral support
- Native C/C++ vision libraries exposed via Python
- Real-time image capture and processing
- Integrated OpenMV IDE with live preview and REPL
- Frozen modules included in firmware (no installs)
- Supports Wi-Fi/BLE networking on compatible boards
- Designed for edge AI and low-power vision systems

Python standard libraries and micro-libraries

- The following standard Python libraries have been “micro-ified” to fit in with the philosophy of MicroPython.
- They provide the core functionality of that module and are intended to be a drop-in replacement for the standard Python library.

- `array` – arrays of numeric data
- `asyncio` — asynchronous I/O scheduler
- `binascii` – binary/ASCII conversions
- `builtins` – builtin functions and exceptions
- `cmath` – mathematical functions for complex numbers
- `collections` – collection and container types
- `errno` – system error codes
- `gc` – control the garbage collector
- `gzip` – gzip compression & decompression
- `hashlib` – hashing algorithms
- `heapq` – heap queue algorithm
- `io` – input/output streams
- `json` – JSON encoding and decoding
- `marshal` – Python object serialization
- `math` – mathematical functions
- `os` – basic “operating system” services
- `platform` – access to underlying platform’s identifying data
- `random` – generate random numbers
- `re` – simple regular expressions
- `select` – wait for events on a set of streams
- `socket` – socket module
- `ssl` – SSL/TLS module
- `struct` – pack and unpack primitive data types
- `sys` – system specific functions
- `time` – time related functions
- `zlib` – zlib compression & decompression
- `_thread` – multithreading support

MicroPython-specific libraries

- `bluetooth` — low-level Bluetooth
- `cryptolib` – cryptographic ciphers
- `deflate` – deflate compression & decompression
- `machine` — functions related to the hardware
- `micropython` – access and control MicroPython internals
- `network` — network configuration
- `openamp` – provides standard Asymmetric Multiprocessing (AMP) support
- `uctypes` – access binary data in a structured way
- `mutex` — mutex module
- `uping` — Ping another computer
- `requests` — Related functions of HTTP client
- `vfs` – virtual filesystem control

Libraries specific to the OpenMV Cam

- `pyb` — functions related to the board
- `stm` — functionality specific to STM32 MCUs
- `sensor` — camera sensor
- `csi` — camera sensors
- class CSI – Camera Sensor Interface
- `image` — machine vision
- `ml` — Machine Learning
- `gif` — gif recording
- `mjpeg` — mjpeg recording
- `audio` — Audio Module
- `display` — display driver
- `fir` — thermal sensor driver (fir == far infrared)
- `tof` — time-of-flight sensor driver
- `tv` — tv shield driver
- `cpufreq` — CPU Frequency Control
- `imu` — imu sensor
- `rpc` — rpc library
- `rtsp` — rtsp library
- `omv` — OpenMV Cam Information
- `gt911` — Touch Screen Driver
- `ft5x06` — Touch Screen Driver
- `tfp410` — DVI/HDMI Controller
- `pca9674a` — I2C Expander Driver
- class PCA9674A – 8-bit I/O expander

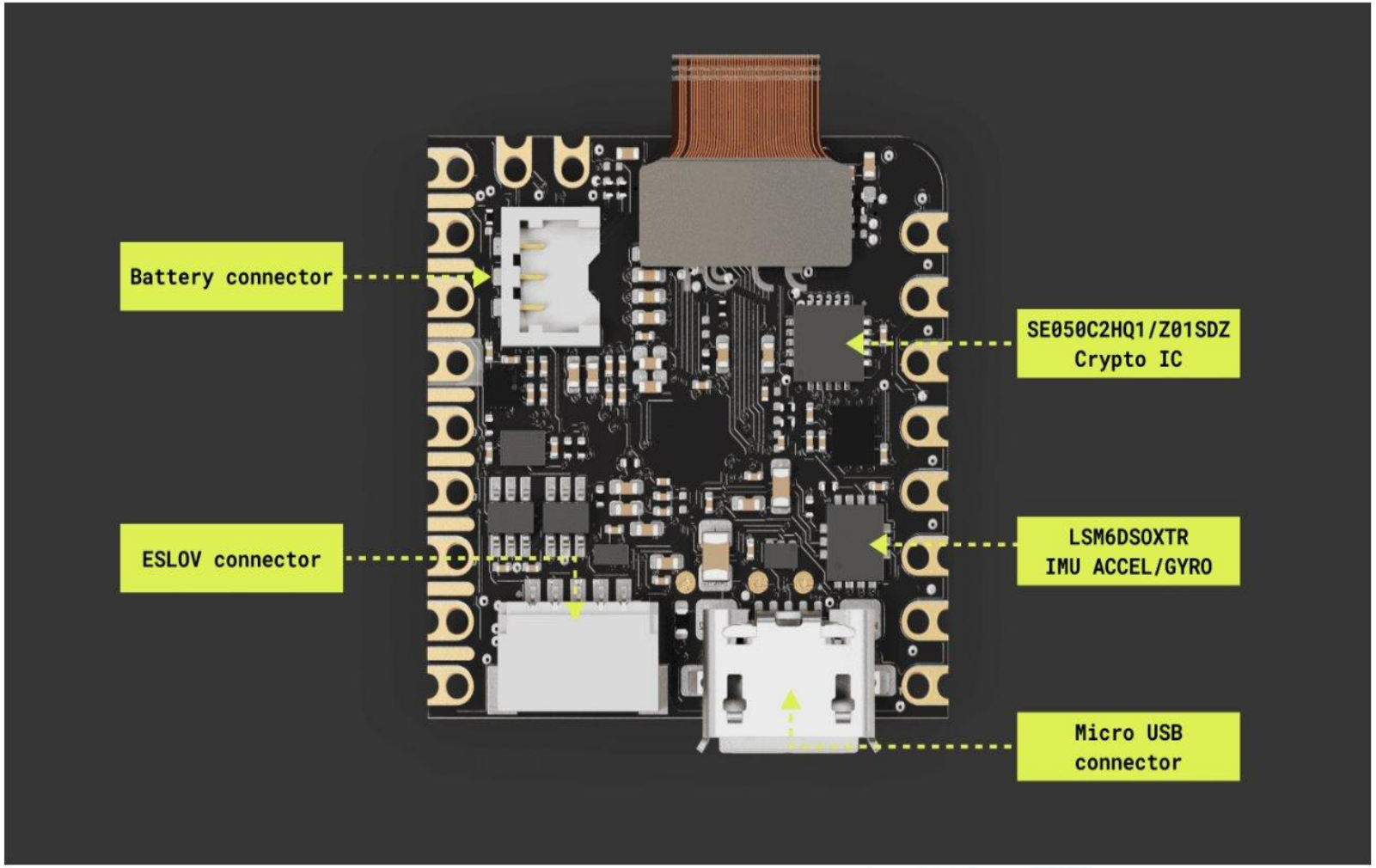
Examples and Getting Started Guide

- https://docs.arduino.cc/tutorials/nicla-vision/getting-started/

Terminology

- **Microcontroller (MCU)**
  – The main chip that runs the program and controls hardware
- **GPIO**
  – Digital pins used to read or write electrical signals
- **Pin**
  – A physical connection on the MCU
  – Can be INPUT or OUTPUT
- **HIGH / LOW**
  – HIGH (1) → 3.3 V
  – LOW (0) → 0 V
- **Pull-Up Resistor**
  – Keeps an input pin at HIGH when not actively driven
- **Peripheral**
  – External or onboard hardware connected to the MCU
  – Example: LED, IMU, camera
- **Communication Protocol**
  – Rules for transferring data between MCU and peripherals
  – Examples: SPI, I2C
- **Chip Select (CS)**
  – A GPIO pin used in SPI to select a specific device

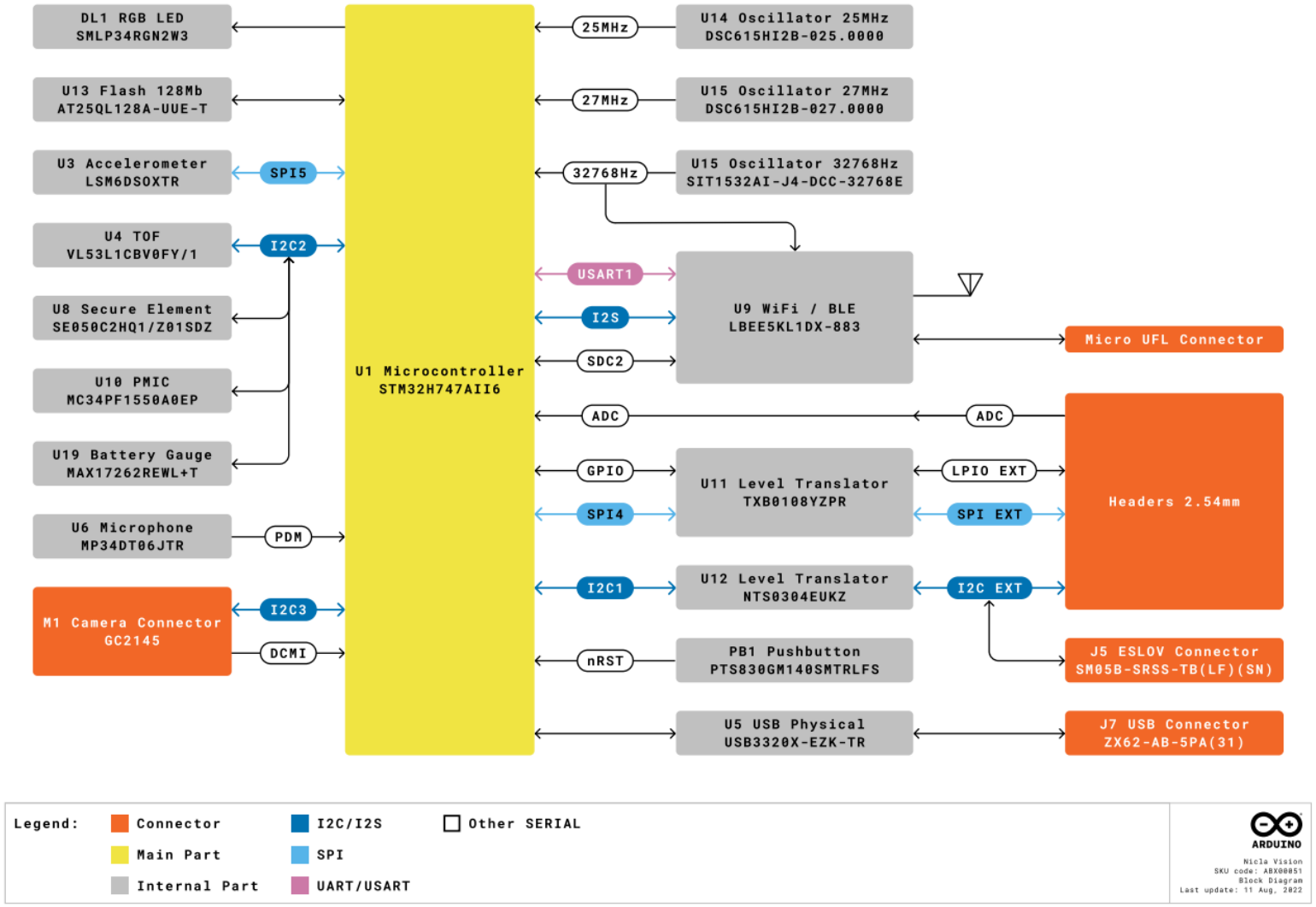
Let's Understand On-Board Sensor



Accessing Pins

Refer to this Schematics to know details about onboard sensors and pins

https://docs.arduino.cc/resources/schematics/ABX00051-schematics.pdf



Nicla Vision Block Diagram

What Is a Pin?

A pin is a physical electrical connection on the microcontroller used to:

- Write signals (turn something ON/OFF)
- Read signals (sense a state or event)
- In MicroPython, pins are controlled using the *Pin* class from the *machine* module.

Writing to a pin means sending a voltage:

- HIGH (1) → 3.3V
- LOW (0) → 0V

Write on Pins

Writing on LED pins

```
from machine import Pin
import time
# Configure the onboard green LED as an output
led = Pin("LED_GREEN", Pin.OUT)
while True:
    led.on()          # Set pin HIGH (LED ON)
    time.sleep(1)
    led.off()         # Set pin LOW (LED OFF)
    time.sleep(1)
```

Exercise

Morse code blinker (pattern programming)

- **Task:** Blink **SOS** using a list like `[200,200,200, 600,600,600, 200,200,200]` (ms), where short = dot, long = dash.
- **Constraint:** Use only one LED and one loop.

Read on Pins

Reading a pin means checking its voltage level:

0 → LOW

1 → HIGH

Why **Pull-Up**?

Keeps the pin at HIGH when not pressed

```
from machine import Pin

import time


# Configure pin PA0 as an input with pull-up
resistor

button = Pin("PA0", Pin.IN, Pin.PULL_UP)


while True:

    state = button.value()  # Read pin state

    print("Button state:", state)

    time.sleep(0.5)
```

Exercise

Convert raw input into a Boolean state

Task

- Read the input pin
- Create a variable called `button_pressed`
- `button_pressed` should be `True` only when the button is actually pressed
- Print only `True` or `False` (not raw pin values)

Exercise

Wrist OrientationDetection UsingGyroscope.

In this exercise, you will use the **gyroscope** to detect **wrist orientation and rotation direction**, and indicate it using **different LED colors**.

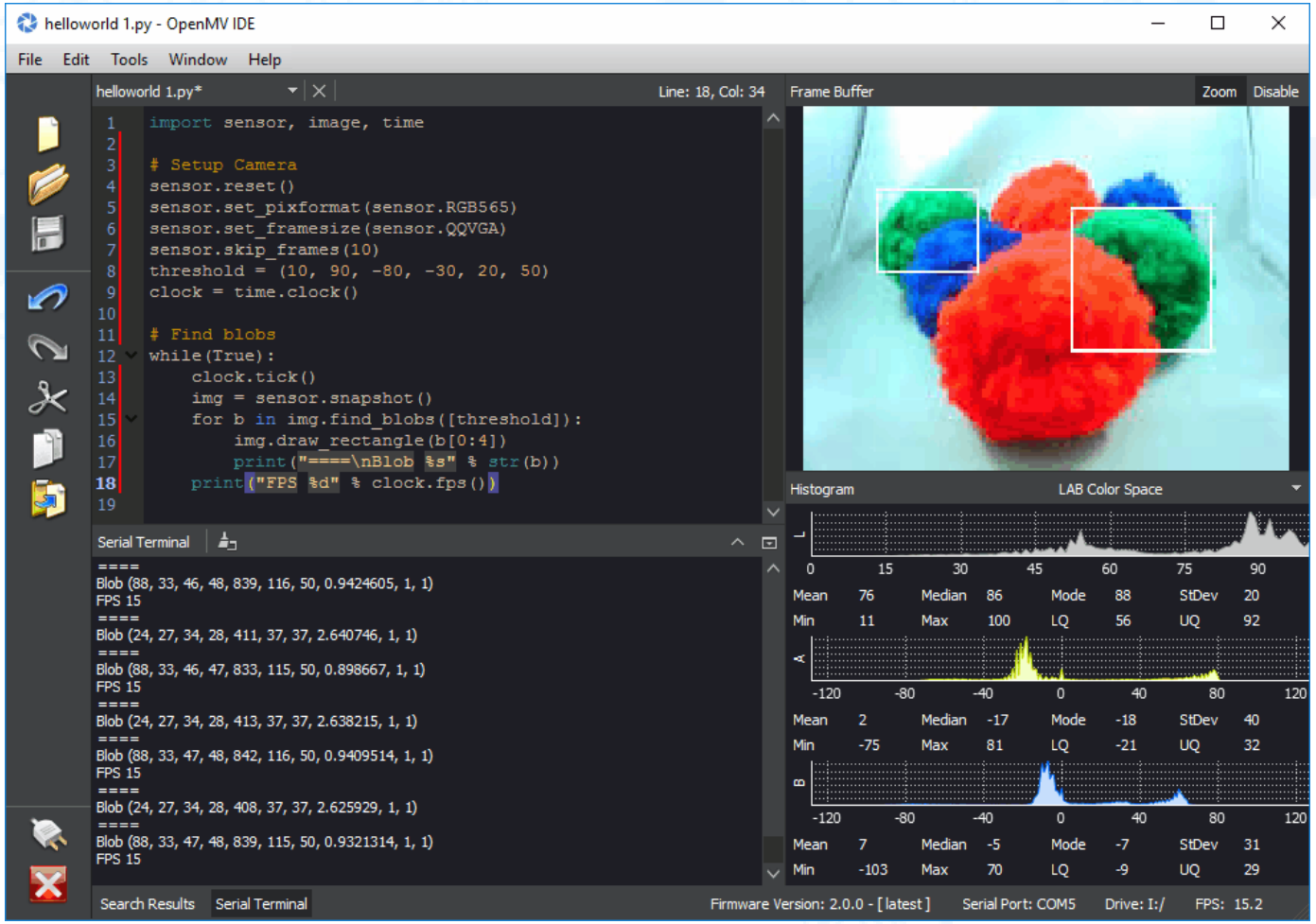
- **Read gyroscope data**
  - Use the IMU gyroscope readings:
  - `gx, gy, gz = imu.gyro()`
  - Values represent angular velocity along X, Y, Z axes
- **Determine dominant rotation axis**
  - At each sample, identify which axis (`gx`, `gy`, or `gz`) has the **largest absolute value**
  - Ignore small values (below a fixed threshold)
- **Map wrist motion to LED color**
  - Rotation about **X-axis** → Turn ON **Green LED**
  - Rotation about **Y-axis** → Turn ON **Blue LED**
  - Rotation about **Z-axis** → Turn ON **Red LED**
  - No significant rotation → All LEDs OFF
- **Single-axis indication**
  - Only **one LED may be ON at a time**
  - Update LEDs only when dominant axis changes
- **Constraints**
  - Use a **fixed angular velocity threshold**
  - Do not use acceleration data
  - Sampling interval: `50 ms`
  - Do not change IMU configuration

Part 3: Software installation and Testing

OpenMV IDE

OpenMV IDE

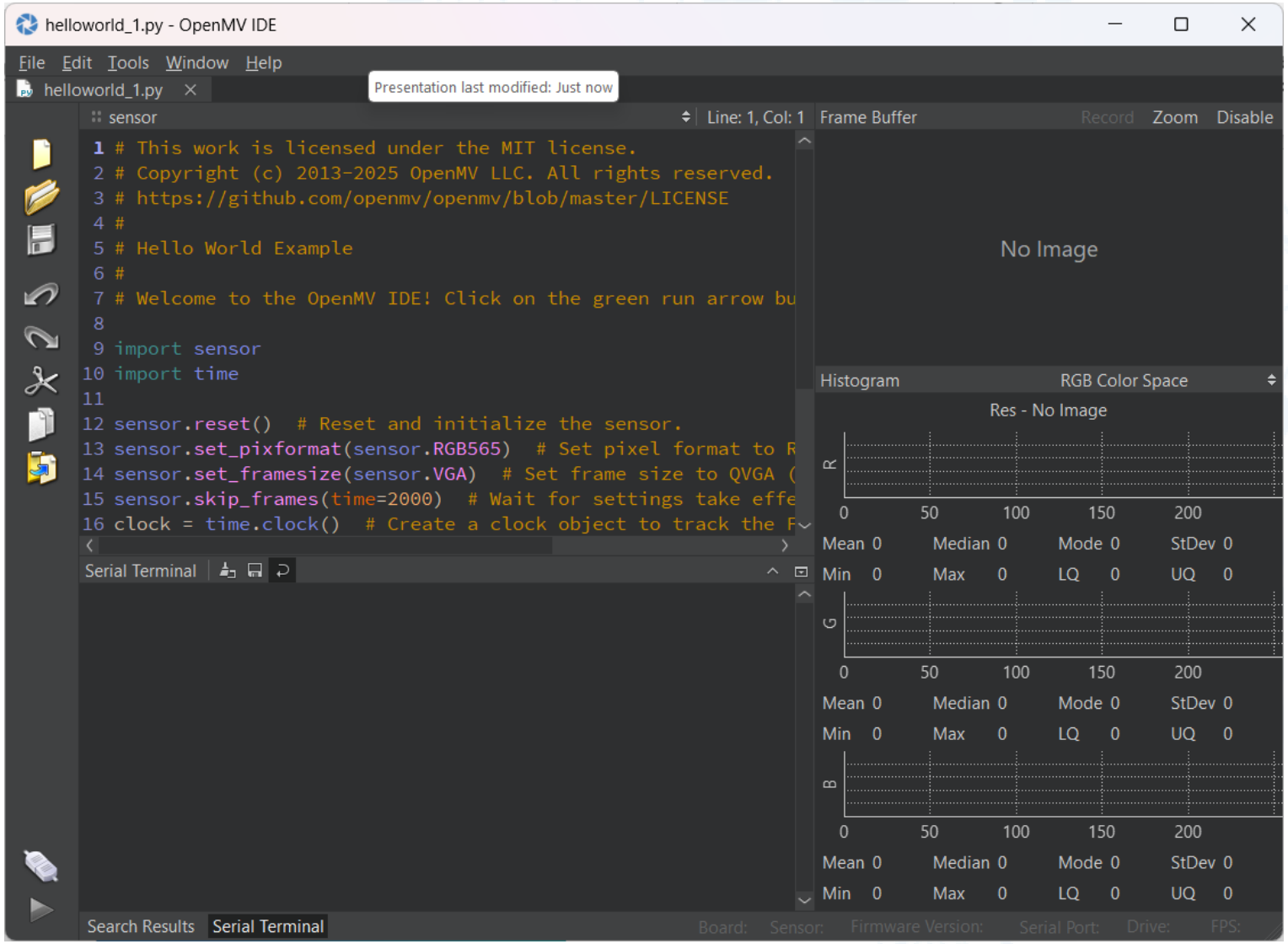
- Code editor with syntax highlighting and auto-completion
- One-click connect/disconnect to OpenMV devices
- Run/stop scripts with inline error highlighting
- Live frame buffer viewer for real-time camera output
- Frame capture and image/video recording for data collection
- Histogram and color tools for image tuning
- Built-in serial terminal and REPL for debugging
- Status bar showing FPS, port, and firmware details
- Utilities for device reset, bootloader, and configuration



OpenMV

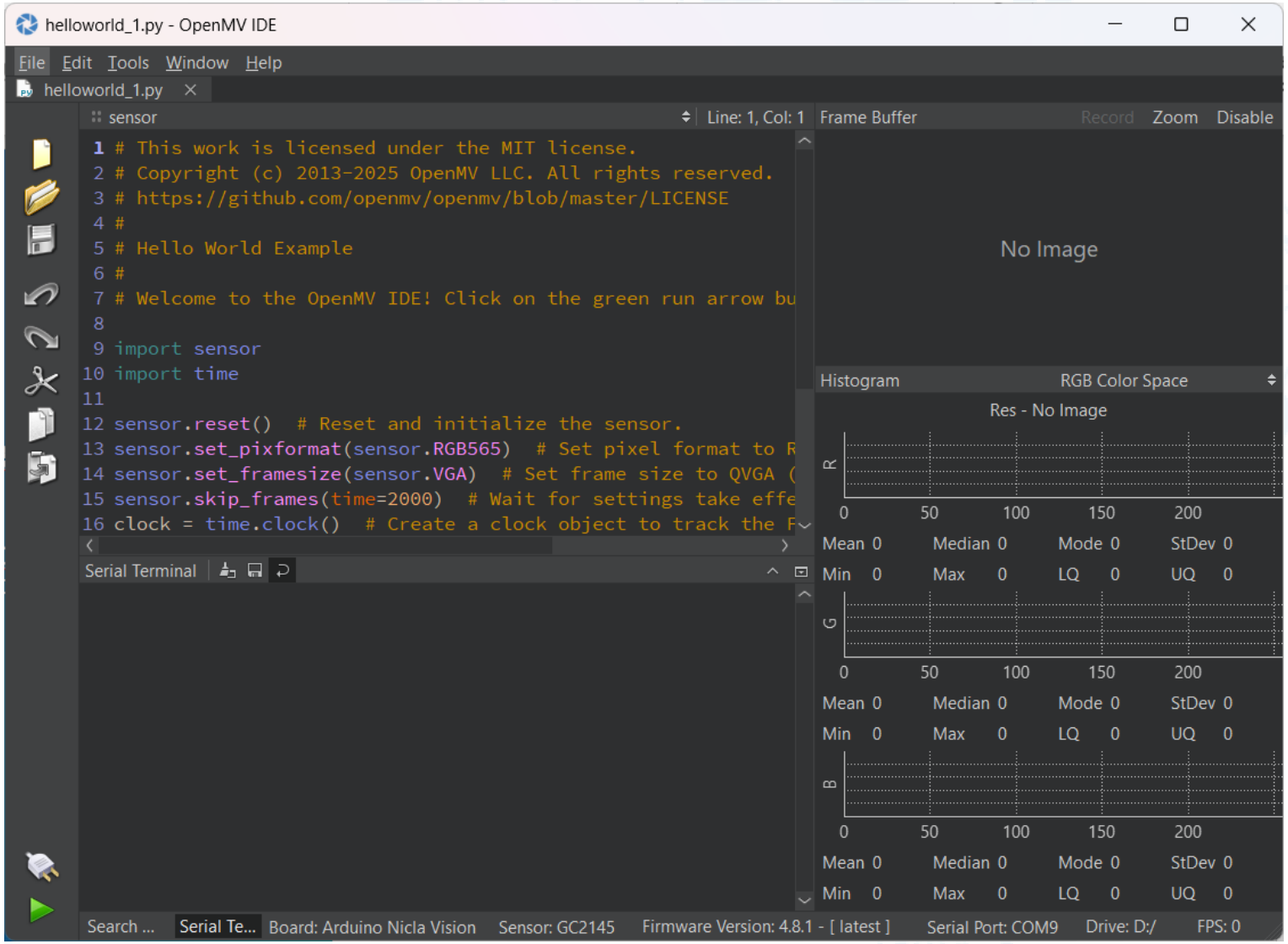
Open OpenMV IDE and connect your Nicla Vision.

Click on the Connect button which is shown on the bottom left.



OpenMV

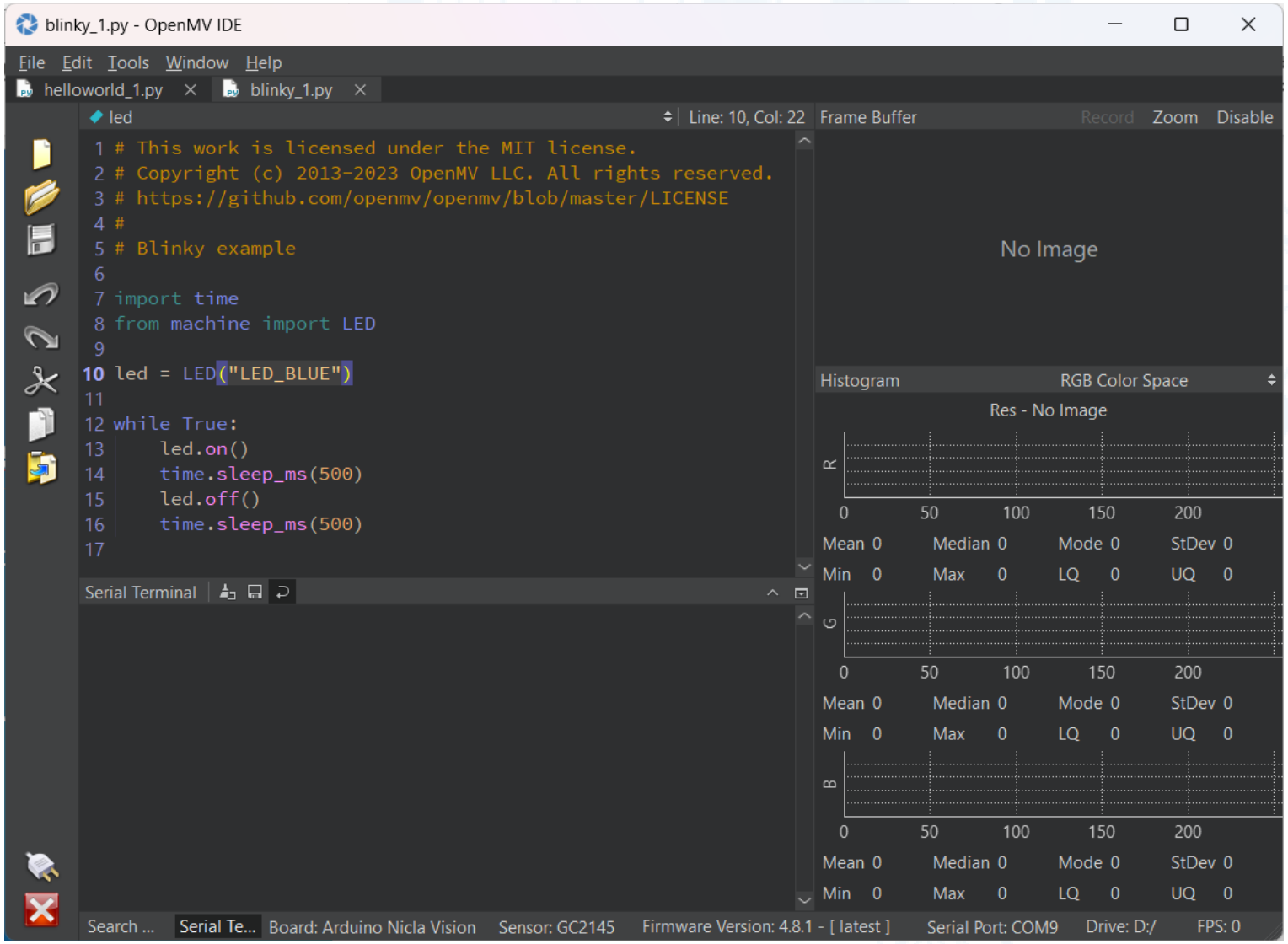
Once connected the green play button will show on the bottom left.



OpenMV

Open the Blink example and run it.

File -> Examples -> HelloWorld -> blinky.py

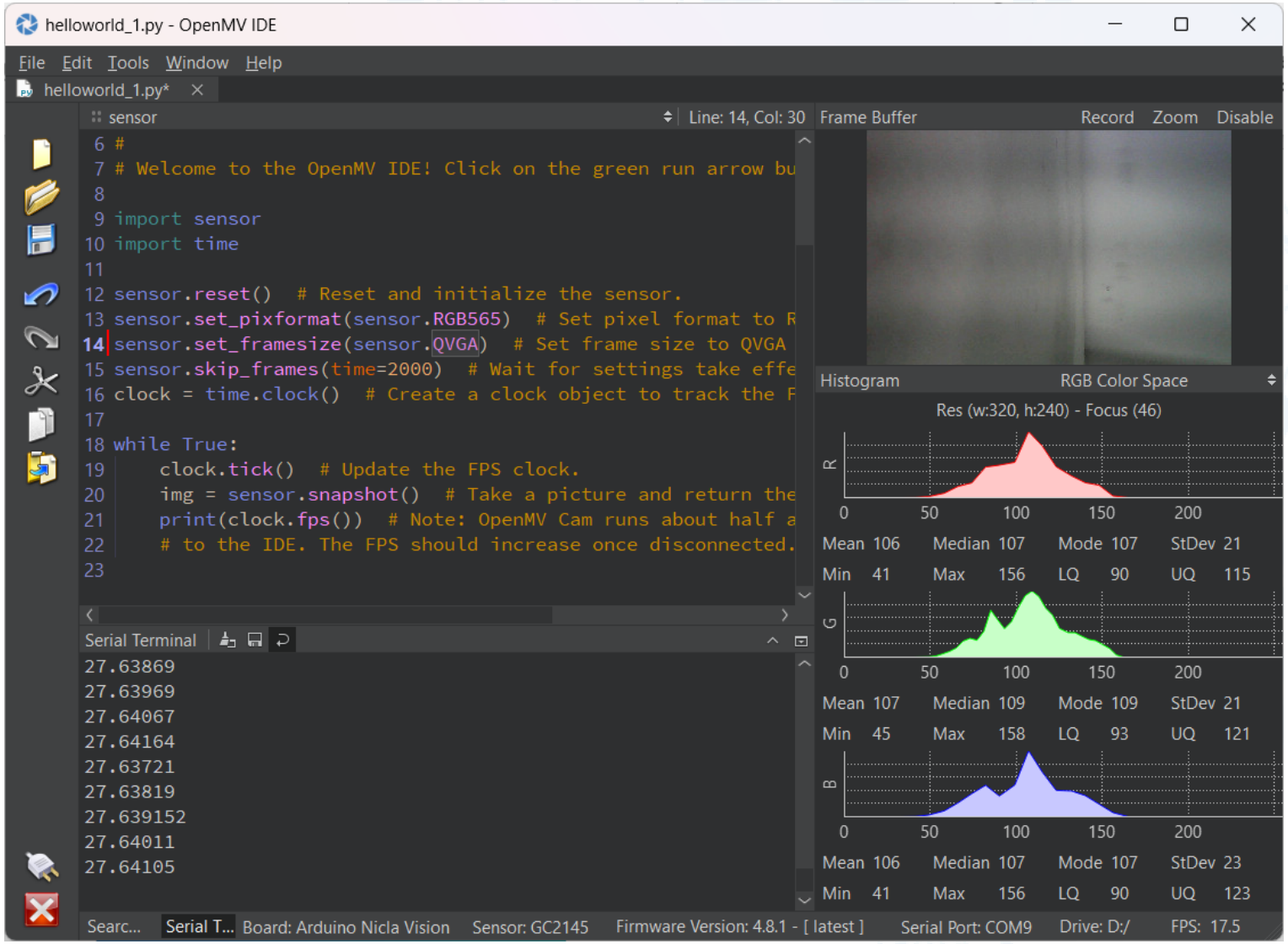


OpenMV

Open the camera example and run it.

File -> Examples -> HelloWorld -> helloworld.py

Change sensor.VGA to sensor.QVGA

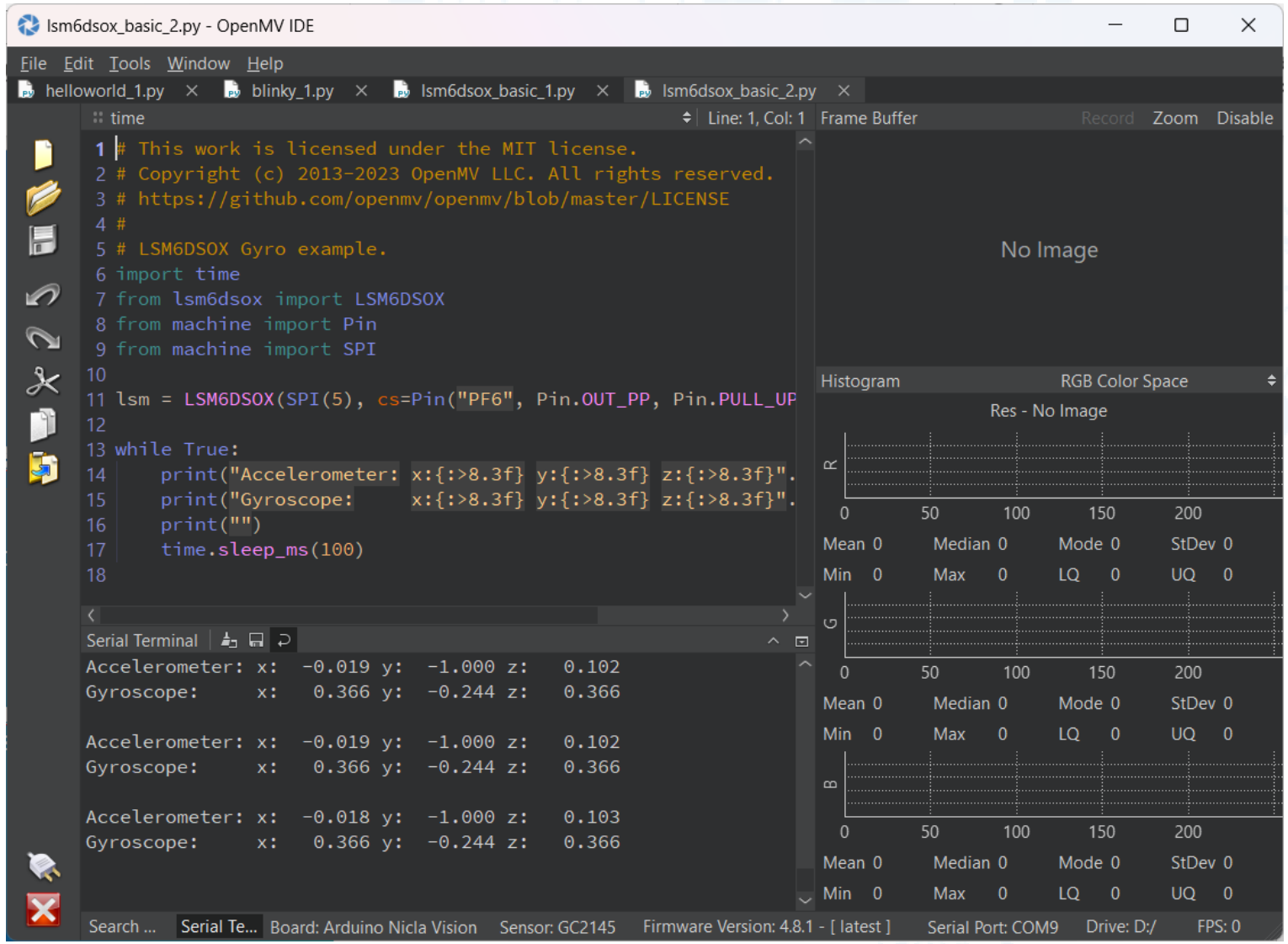


OpenMV

Open the IMU example and run it.

File -> Examples -> Sesnors -> lsm6dsox_basic.py

Try other examples.



Source: https://premioinc.com